

HOCHSCHULE
HANNOVER
UNIVERSITY OF
APPLIED SCIENCES
AND ARTS

–
*Fakultät IV
Wirtschaft und
Informatik*

Convolution und ihre Anwendung in der digitalen Audiobearbeitung

Seminarfach: Audio DSP

Julian Mueller Tom Nir
25. Mai 2026



1 Grundlagen: Audio DSP

2 Convolution

3 Implementierung

4 Literatur



Sampling I

- Abtastung stetiger Signale $\implies$ Folge diskreter Amplitudenwerte

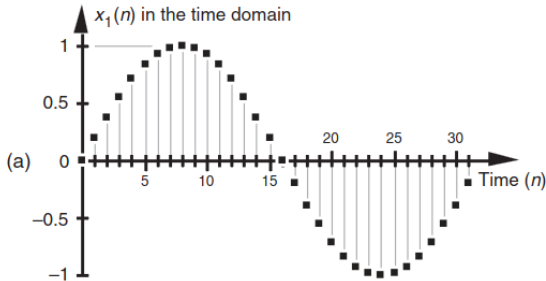


Abbildung: Horiz. Achse: Zeitbereich (n -ter Samplewert) Vert. Achse: Amplitude [2, Kap. 2]



Zeit- und Frequenzdomäne I

- Zeitdomäne: Horizontale -> diskrete Zeitindizes; Vertikale -> Amplitude
- Frequenzdomäne: Horizontale -> Frequenz in Hz ; Vertikale -> Magnitude

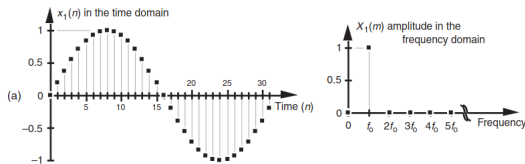


Abbildung: Gegenüberstellung: Zeit-/Frequenzdomäne einer Sinuswelle Lyons, 2011¹



Zeit- und Frequenzdomäne II

- Umwandlung eines Signals in Zeitdomäne in Frequenzdomäne mithilfe der *Fourier Transformation*:

$$X[m] = \sum_{n=0}^{N-1} x[n] \cdot e^{\frac{-i2\pi \cdot nm}{N}} \quad (1)$$

¹2, Kap. 2.

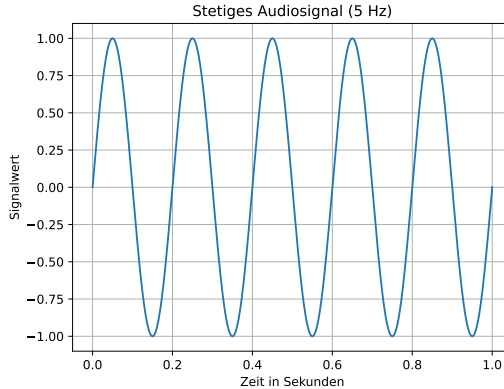


Signale I

- Signale beschreiben, wie ein Parameter sich in Abhängigkeit eines anderen ändert
- x-Achse: Zeit/Frequenz
- y-Achse: Signalwert (abhängige Variable)
- Notation: $x[n]$ für diskrete Signale, Signalwert x an Index n



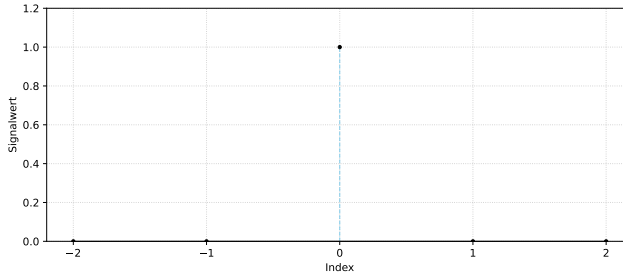
Signale II



Der Einheitsimpuls I

- Genauer: Einheits-Sample-Folge

$$\delta[n] = \begin{cases} 0, & n \neq 0 \\ 1, & n = 0 \end{cases} \quad (2)$$



- System bildet Eingangssignal $x[n]$ auf Ausgangssignal $y[n]$ ab
- T : Transferfunktion des Systems

$$T : x[n] \mapsto y[n]$$



Lineare Zeit-Invariante Systeme I

Lineare Systeme:

$$T(x_1[n] + x_2[n]) = T(x_1[n]) + T(x_2[n]) = y_1[n] + y_2[n] \quad (3)$$

$$T(a \cdot x[n]) = a \cdot T(x[n]) = a \cdot y[n] \quad (4)$$

- Summe der Ausgangssignale ist gleich Ausgangssignal bei vorheriger Summierung der Eingangssignale.



Lineare Zeit-Invariante Systeme II

Zeit-Invariante Systeme:

- für jedes $n_0 \in \mathbb{N}$ und jedes Eingangssignal $x[n]$ mit $x'[n] = x[n - n_0]$ wird das entsprechende Ausgangssignal y mit $y'[n] = y[n - n_0]$ erzeugt.
- Verzögertes Eingangssignal $\rightarrow$ verzögertes, sonst gleiches, Ausgangssignal.



Lineare Zeit-Invariante Systeme III

Systeme mit beiden Eigenschaften: *Lineare Zeit-Invariante Systeme*

- Werden vollständig durch ihre Impulsantwort charakterisiert.
- Kurzschreibweise: LTI-System



1 Grundlagen: Audio DSP

2 Convolution

3 Implementierung

4 Literatur



Convolution-Summe I

Jedes diskrete Signal kann als Summe skalierten, verzögerter Einheitsimpulse dargestellt werden [3].



Convolution-Summe II

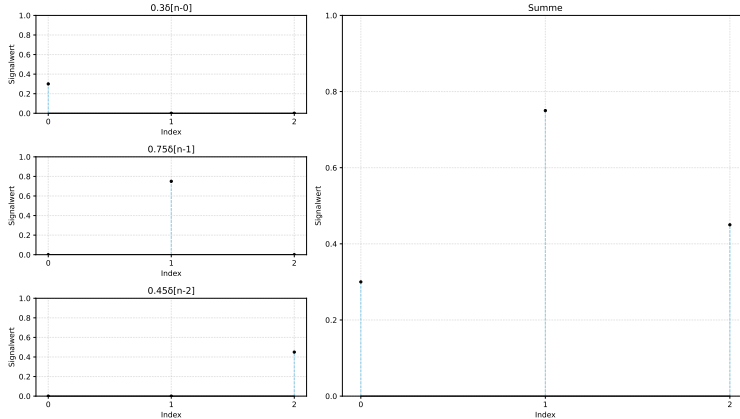


Abbildung: Summe dreier skalierten, verzögerter Einheitsimpulse



Convolution-Summe III

Diskretes Signal als Summe skalierten, verzögerter Einheitsimpulse:

$$x[n] = \sum_{k=-\infty}^{\infty} x[k]\delta[n - k] \quad (5)$$



Convolution-Summe IV

Anwenden einer Transferfunktion T auf ein Signal:

$$y[n] = T(x[n]) = T \left(\sum_{k=-\infty}^{\infty} x[k] \delta[n - k] \right) \quad (6)$$



Convolution-Summe V

Für lineares System gilt:

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]T(\delta[n-k]) = \sum_{k=-\infty}^{\infty} x[k]h_k[n] \quad (7)$$

- h_k : Transferfunktion eines Systems bei Eingangssignal $\delta[k]$ an Index n
- System könnte unterschiedlich auf Einheitsimpulse an verschiedenen Indizes reagieren



Convolution-Summe VI

Für Lineares Zeit-Invariantes System gilt:

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k] \quad (8)$$

- Convolution-Summe
- Wann $h[n]$ bestimmt wurde ist nicht mehr relevant!
- Kurzschreibweise: $y[n] = x[n] * h[n]$



Moving Average Filter I

Anschauliches Beispiel für Convolution-Summe: Moving Average Filter
(Low-Pass FIR-Filter)

- Werte von $y[n]$ berechnet aus Mittelwert einiger Werte aus $x[n]$



Moving Average Filter II

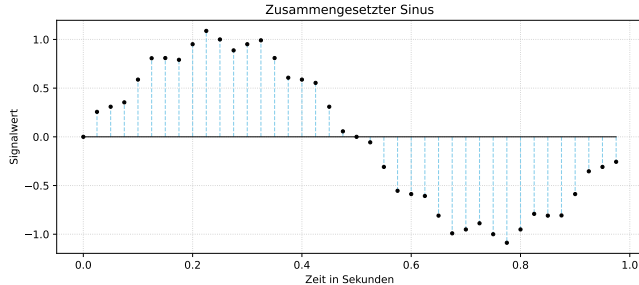


Abbildung: Ein diskretes Signal, zusammengesetzt aus Sinusschwingungen mit den Frequenzen $f_0 = 1$ Hz, $f_1 = 10$ Hz, und $f_2 = 20$ Hz



Moving Average Filter III

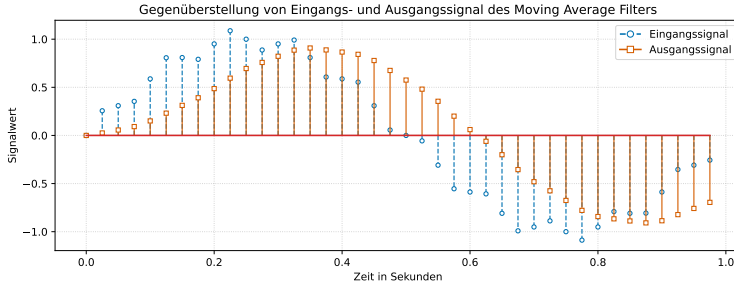


Abbildung: Dasselbe diskrete Signal, Ausgangssignal des Moving Average Filters überblendet



Moving Average Filter IV

Es werden m Samples summiert, und das Ergebnis mit $\frac{1}{m}$ multipliziert:

$$y[n] = \frac{1}{m} \sum_{k=0}^{m-1} x[n-k] \iff y[n] = \sum_{k=0}^{m-1} \frac{1}{m} x[n-k]. \quad (9)$$

[2, S. 172].



Moving Average Filter V

Koeffizienten $\frac{1}{m}$ können als Folge der Länge m mit dem Wert $\frac{1}{m}$ an jedem Index interpretiert werden. Zum Beispiel für $m = 5$:

$$y[n] = \sum_{k=0}^4 h[k]x[n-k] \text{ mit } h = \left(\frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}\right). \quad (10)$$

$h[n]$ ist die Impulsantwort des FIR-Filters! [2, S. 172f]



Moving Average Filter VI

Die Impulsantwort ist das Ausgangssignal des Systems (hier Moving Average Filter) bei einem Eingangssignal $\delta[n]$.



Convolution-Theorem I

Für Signale $x[n]$, $y[n]$, $h[n]$ seien $X(e^{i\omega})$, $H(e^{i\omega})$ und $Y(e^{i\omega})$ die Repräsentationen der Signale in Frequenzdomäne.

Es gilt:

$$Y(e^{i\omega}) = X(e^{i\omega}) \cdot H(e^{i\omega}). \quad (11)$$

- Die Berechnung der Convolution-Summe zweier Signale in Zeitdomäne ist äquivalent zum Produkt der Signale in Frequenzdomäne.

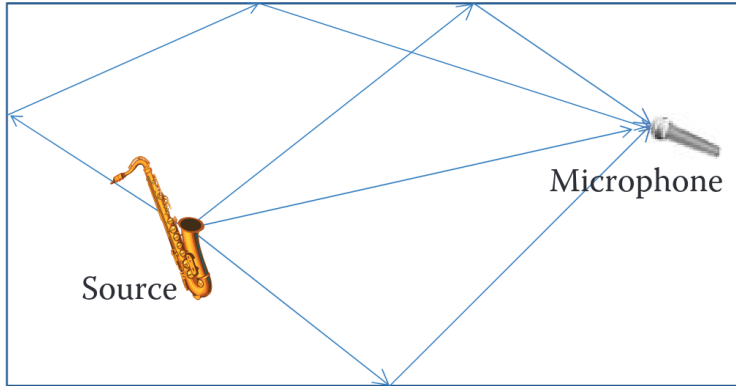


LTI-Systeme in der realen Welt I

Gibt es LTI-Systeme in der realen Welt?



LTI-Systeme in der realen Welt II



[4, Reiss, McPherson 2015]



LTI-Systeme in der realen Welt III



[1, Farina 2007a]



LTI-Systeme in der realen Welt IV

- Räume
- Mikrofone
- Lautsprechersysteme (Speaker-Kabinetts für Gitarrenverstärker, Studiomonitore, Kopfhörer)
- noch viele mehr, über verschiedene Ingenieurwissenschaften verteilt



LTI-Systeme in der realen Welt V

Benötigt wird die Impulsantwort des Systems.

Beispiel Raumhall:

- Abspielen eines Impuls im Raum
- Aufnehmen des Raumschalls



LTI-Systeme in der realen Welt VI

Aufnahme besteht aus Impuls, und der Transferfunktion des Raumes angewandt auf den Impuls.

- Impulsantwort muss aus Aufnahme berechnet werden
- System nicht perfekt linear: Verstärker und Lautsprecher leicht nonlinear
- System nicht perfekt Zeit-invariant: Turbulenzen in Raumlauft verändern Schallgeschwindigkeit



Messung von Impulsantworten I

Verschiedene Methoden:

- Minimum Length Sequence (MLS): digital erzeugtes Rauschen mit praktischer Eigenschaft
- Time Delay Spectroscopy (TDS): linearer Sinus-Sweep
- Logarithmic/Exponential Sine Sweep (LSS/ESS):
logarithmischer/exponentieller Sinus-Sweep



Messung von Impulsantworten II

LSS/ESS-Methode für Convolution Reverb am besten geeignet.

- Kann mit leichten Nonlinearen Eigenschaften umgehen
- Kein Rauschen durch Zeitvariante Eigenschaften
- Deconvolution durch Convolution mit inversem Sinus-Sweep (mehr oder weniger)



Laufzeitanalyse I

```
1 Algorithm directConvolution(inSignal, N, irSignal, M)
2   outLen <- N + M - 1      // O(1)
3   for i <- 0 to outLen - 1  // O(N + M + 1)
4     for k <- 0 to M - 1     // O(M)
5       if i-k < 0 or n <= i-k // O(1)
6         continue
7
8       outSignal[i] <- outSignal[i]
9       + irSignal[k] * inSignal[i-k] // Worst-Case: O(1)
10    done
11  done
12  return outSignal
```

Abbildung: Pseudocode: Implementierung der diskreten Convolution



Laufzeitanalyse II

- Worst-Case Laufzeitklasse:

$$\mathcal{O}(1 + (N + M - 1) \cdot (M + 2)) = \mathcal{O}(M^2 + NM - M + 2N - 1) \quad (12)$$

- ... vereinfacht sich lt. Polynomregel zu:

$$\mathcal{O}(M^2 + NM) \quad (13)$$

- ... es gilt aber

$$N \gg M \implies N \cdot M > M^2 \quad (14)$$

- ... woraus folgt: $\mathcal{O}(N \cdot M)$



Laufzeitanalyse III

```
1 Algorithm fftConvolution(inSignal, N, irSignal, M)
2   outLen <- N + M - 1 // O(1)
3   fftLen <- outLen    // O(1)
4
5   // zeroPad() füllt die Listen bis fftLen mit nullen auf
6   zeroPad(inSignal, fftLen) // O(n)
7   zeroPad(irSignal, fftLen) // O(n)
8
9   inSpectrum <- FFT(inSignal, N, fftLen) // O(n log(n))
10  irSpectrum <- FFT(irSignal, M, fftLen) // O(n log(n))
11
12  for f <- 0 to fftLen - 1 // Insg. O(n)
13    outSpectrum[f] <- inSpectrum[f] * irSpectrum[f]
14  done
15
16  outTimeDomain <- IFFT(outSpectrum, fftLen) // O(n log(n))
17
18  // real() isoliert den Realteil des komplexen IFFT-Outputs
19  for i <- 0 to outLen - 1 // Insg. O(n)
20    outSignal[i] <- real(outTime[i])
21  done
22
23  return outSignal
```

Abbildung: Pseudocode: Implementierung der Convolution mithilfe der *Fast Fourier Transformation*



Laufzeitanalyse IV

- Laufzeit:

$$\mathcal{O}(3(N \cdot \log(N)) + 4N + 2) \in \mathcal{O}(N \cdot \log(N)) \quad (15)$$



1 Grundlagen: Audio DSP

2 Convolution

3 Implementierung

4 Literatur



Direkte Convolution I

```
20  std::vector<float> convolution(std::vector<float>& input, std::vector<float>& ir) {
21
22      unsigned long inputLen {input.size()};
23      unsigned long irLen {ir.size()};
24      unsigned long outputLen {inputLen + irLen - 1};
25
26      std::vector<float> output (outputLen, 0.0f); //-- Mit 0.0f initialisieren
27
28      for (int i = 0; i < outputLen; i++) {
29          for (int k = 0; k < irLen; k++) {
30              if (i - k < 0 || inputLen <= i - k) {
31                  continue;
32              }
33              output[i] += ir[k] * input[i - k];
34          }
35      }
36
37      return output;
38  }
```

Abbildung: Implementierung der direkten Convolution in C++



Direkte Convolution II

[DEMO]
2

²5, Benutzte Impulsantwort (Sounddatei).



Laufzeitmessung I

```
• time ./run
JUCE v8.0.12
Eingangssignal geladen! Anzahl Samples: 416930
Impulsantwort geladen! Anzahl Samples: 176584
Starting convolution ...
[#####-] 99%
Normalisiere Ausgangssignal ...
Output File created successfully!

real    8m55,022s
user    8m54,883s
sys     0m0,024s
```

Abbildung: Messung des gezeigten Codes 8 unter Verwendung des Linux-Tools *time*

- Samplerate beider Signale: 44100 Hz
- Eingangssignal: Etwa 10-sekündiges Gitarrensinal
- Impulsantwort: Etwa 5 Sekunden
- Laufzeitklasse: $\mathcal{O}(N \cdot M)$ (vgl. Folie 35)



Laufzeitmessung II

- Anzahl der Berechnungen für das ≈ 15 -sekündige Ausgangssignal:

$$\begin{aligned} G_{\text{Berechnungen}} &= (N + M - 1) \cdot M \\ &= (416.930 + 176.584 - 1) \cdot 176.584 = 104.804.899.592 \end{aligned} \quad (16)$$

- Entspricht ≈ 7 -Milliarden Berechnungen, die der Prozessor in einer Echtzeitanwendung ausführen müsste.



Fazit I

- LTI-Systeme
- Convolution
- Impulsantworten von Systemen
- Laufzeitanalyse: Direkte Convolution vs. mithilfe der FFT
- Laufzeiten naiver Implementierungsansätze (DFT oder direkte Convolution) ungeeignet
- Effizienter Lösungsansatz in der Praxis: Fast Fourier Transformation



1 Grundlagen: Audio DSP

2 Convolution

3 Implementierung

4 Literatur



Literaturverzeichnis I

- [1] Angelo Farina. "Advancements in impulse response measurements by sine sweeps". en. In: (2007).
- [2] Richard G. Lyons. *Understanding Digital Signal Processing*. third. Pearson Education, 2011. ISBN: 978-0-13-702741-5.
- [3] Alan V. Oppenheim, Ronald W. Schafer und John R. Buck. *Discrete-time signal processing*. 2nd ed. Upper Saddle River, N.J: Prentice Hall, 1999. ISBN: 978-0-13-754920-7.
- [4] Joshua D. Reiss und Andrew P. McPherson. *Audio Effects: Theory, Implementation and Application*. Hoboken: CRC Press, 2015. ISBN: 978-1-4665-6028-4.
- [5] Aleksey Vaneev. *Free impulse responses*. 2026. URL: <https://www.voxengo.com/impulses/> (besucht am 25.05.2026).

